

STOCKFLOW

Documentacao do prototipo - Sistema de Controle de Estoque

Tecnologias	Python, Tkinter, MySQL, Matplotlib
Objetivo	Controlar produtos e movimentacoes de estoque em uma aplicacao desktop simples.
Login inicial	usuario: admin senha: admin123
Arquivos principais	main.py, database.py, login.py, gui.py, logger.py

1. Introducao

O StockFlow e um prototipo de sistema desktop para controle de estoque. A aplicacao permite autenticar um usuario, cadastrar produtos, consultar registros, editar e excluir produtos, registrar entradas e saidas e visualizar um grafico de quantidade em estoque. O foco foi manter uma implementacao pequena, modular e facil de demonstrar em sala.

2. Atendimento aos criterios do trabalho

Criterio	Como foi atendido
Multiplas janelas	Janela de login e janela principal do sistema; formularios sao abertos como janelas secundarias.
MySQL	DatabaseManager cria/acessa o banco estoque_db e trabalha com usuarios, produtos e movimentacoes.
CRUD completo	Create, Read, Update e Delete de produtos.
Grafico	Matplotlib gera grafico de barras com a quantidade de cada produto.
Log de erros	Logger grava execucoes em error.log com data, hora e detalhes.
POO	As classes DatabaseManager, LoginWindow, MainWindow e Logger organizam responsabilidades.
Biblioteca extra	Matplotlib e mysql-connector-python.
Documentacao	Este PDF explica arquitetura, banco, execucao e roteiro de demonstracao.

3. Arquitetura do projeto

A separacao em modulos reduz a quantidade de responsabilidades em cada arquivo. O fluxo principal e: main.py inicia o programa -> LoginWindow autentica -> MainWindow apresenta as funcionalidades -> DatabaseManager realiza persistencia no MySQL. O Logger pode ser usado por qualquer parte que precise registrar uma execucao.

Classe / arquivo	Responsabilidade principal
DatabaseManager / database.py	Conectar ao MySQL, criar estrutura e executar operacoes de produtos, usuarios e movimentacoes.
LoginWindow / login.py	Exibir formulario de login e validar usuario/senha.
MainWindow / gui.py	Construir a janela principal, dashboard, CRUD e movimentacoes.
Logger / logger.py	Registrar erros em arquivo.
main.py	Configurar logger, inicializar banco e iniciar Tkinter.

4. Banco de dados

O banco estoque_db possui tres tabelas. A tabela usuarios guarda o acesso ao sistema; produtos guarda o cadastro; movimentacoes registra entradas e saidas relacionadas a um produto.

Tabela	Campos principais	Funcao
usuarios	id, usuario, senha	Autenticacao.
produtos	id, nome, categoria, quantidade, preco, estoque_minimo	Cadastro e saldo atual do estoque.
movimentacoes	id, produto_id, tipo, quantidade, data_hora	Historico de entradas e saidas.

As senhas do aplicativo sao comparadas por SHA-256. O banco tambem pode ser criado manualmente executando banco.sql.

5. Operacoes CRUD

Create: o formulario Novo produto valida os campos e grava com INSERT. Read: a tela Produtos consulta os registros com SELECT e apresenta os dados em uma Treeview. Update: o produto selecionado e alterado com UPDATE. Delete: o produto selecionado e removido com DELETE, apos confirmacao do usuario.

6. Movimentacoes de estoque

Uma entrada aumenta a quantidade do produto e registra uma linha em movimentacoes. Uma saida diminui o estoque. Antes da saida, o sistema verifica se ha unidades suficientes; caso contrario, a operacao e bloqueada. A transacao usa commit/rollback para evitar que a movimentacao e a quantidade fiquem inconsistentes.

7. Grafico

O dashboard consulta os produtos e usa Matplotlib para montar um grafico de barras com a quantidade atual de cada item. O objetivo e demonstrar o uso de uma biblioteca adicional de forma diretamente relacionada ao problema do estoque.

8. Tratamento de erros e log

Falhas de banco e outras excecoes inesperadas sao capturadas nas areas principais da interface e enviadas ao Logger. O arquivo error.log registra data, hora, mensagem e o rastreamento da execucao quando disponivel.

9. Execucao

1) Instalar Python. 2) Ativar ambiente virtual. 3) Executar pip install -r requirements.txt. 4) Conferir DB_CONFIG em database.py. 5) Garantir que o MySQL esteja ativo. 6) Executar python main.py. O programa tenta criar automaticamente estoque_db, tabelas e dados iniciais.

Comandos no PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python main.py
```

10. Roteiro para a apresentacao

- Abrir o sistema e explicar rapidamente o objetivo.
- Fazer login com admin/admin123.
- Mostrar o dashboard e explicar os tres indicadores e o grafico.
- Abrir Produtos e executar cadastrar, listar, editar e excluir.

- Abrir Movimentacoes e registrar uma entrada e uma saida.
- Tentar uma saida acima do estoque para demonstrar a validacao.
- Mostrar error.log e explicar a classe Logger.
- Apresentar a estrutura dos arquivos e as principais classes.

11. Observacao sobre personalizacao

A logica esta separada da aparencia. Para personalizar, o grupo pode alterar as cores e fontes em THEME, textos dos botoes, tamanhos das janelas, espacamentos, posicionamento e o nome STOCKFLOW sem precisar mudar as consultas do MySQL. O arquivo gui.py e o principal ponto de personalizacao visual.

12. Exemplos de uso

O pacote inclui um roteiro de demonstracao pronto para gerar os exemplos reais em sala. Como a aplicacao depende de um servidor MySQL local, os prints das telas em execucao devem ser capturados no computador usado na apresentacao e adicionados a este PDF caso o professor exija evidencias visuais.